Doctor Agent Slide & Naskah Presentasi

Setiap halaman berisi satu slide beserta kata-kata yang diucapkan untuk slide itu. 15 slide, ±15–18 menit.

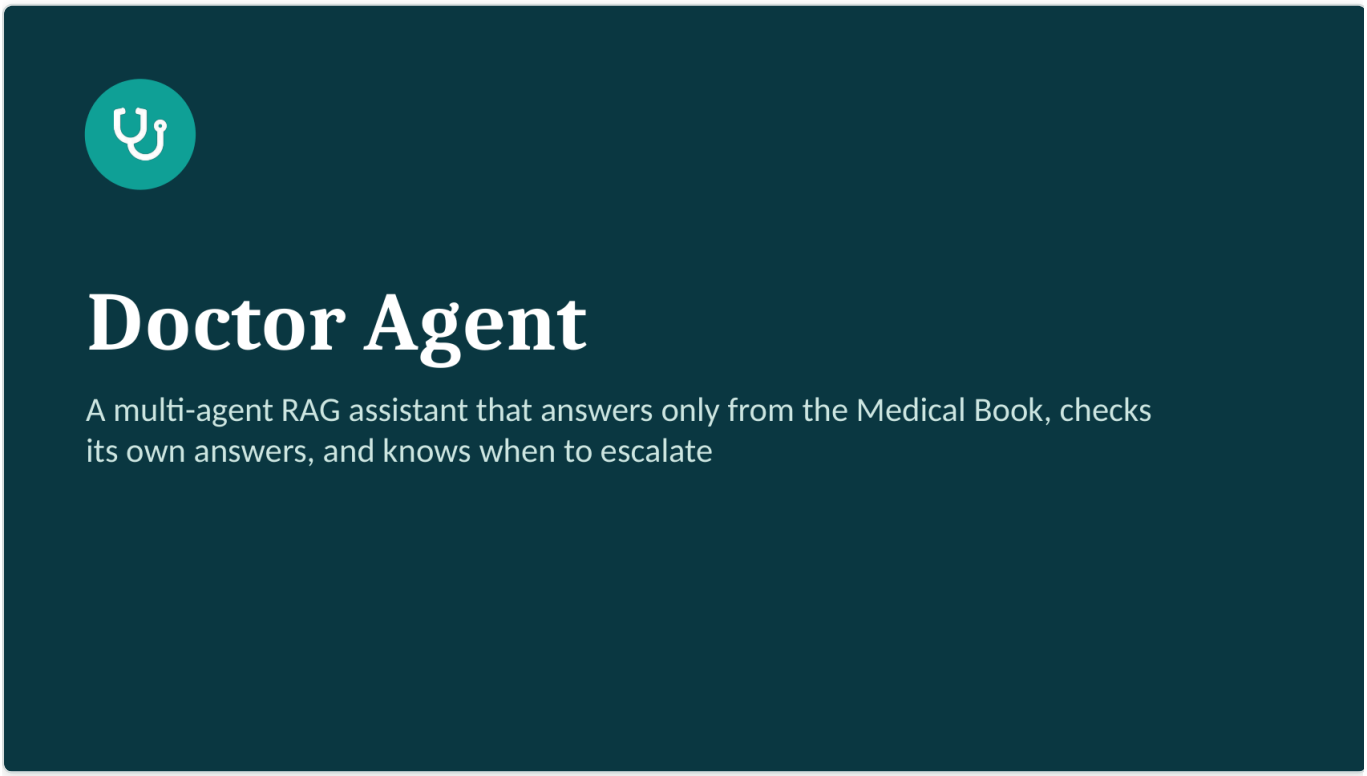
Role AI · Multi-Agent Technical Test

Cara memakai

Total sekitar **15–18 menit** (±1 menit per slide). Kata yang **ditebalkan** adalah poin utama yang sebaiknya tidak terlewat. Teks *[dalam kurung miring]* adalah petunjuk gerakan, bukan untuk diucapkan.

Bagian	Slide	Perkiraan waktu
Pembuka + agenda	1–2	1,5 menit
1 · Approach and methodology	3–7	6 menit
2 · Model architecture and parameters	8–9	2 menit
3 · Evaluation metrics and results	10–12	3,5 menit
4 · Observations and challenges	13	2 menit
5 · Screenshots + penutup	14–15	2 menit

Doctor Agent



"Selamat pagi/siang. Saya Rahmad, dan hari ini saya akan mempresentasikan **Doctor Agent**: asisten medis multi-agent berbasis RAG.

Ada tiga hal yang membedakan sistem ini dari chatbot biasa. Pertama, ia **hanya menjawab dari Medical Book** yang diberikan. Kedua, ia **memeriksa jawabannya sendiri** sebelum ditampilkan. Ketiga, ia **tahu kapan harus berkata 'saya tidak yakin'** dan menyarankan pengguna ke dokter."

Agenda

What this presentation covers

1 · Approach and methodology
Strategy, the four agents, chunking, hybrid retrieval, evaluate-then-decide

2 · Model architecture and parameters
Models per agent, embeddings, index, fusion, thresholds, cost mix

3 · Evaluation metrics and results
27-question test set, metrics, results, retrieval ablation

4 · Observations and challenges
A bug the evaluation caught, cross-lingual retrieval, LLM-as-judge, latency

5 · Screenshots of the Streamlit interface
A checked answer, the agent trace, live progress, an escalation

"Presentasi ini saya susun mengikuti lima poin yang diminta di soal: **pendekatan dan metodologi**, **arsitektur model dan parameter**, **metrik evaluasi dan hasil**, **temuan dan tantangan**, dan terakhir **screenshot aplikasi Streamlit**."

Four agents, one job: trustworthy answers

1 · APPROACH AND METHODOLOGY

Four agents, one job: trustworthy answers

Objective: answer medical questions from the “Medical Book” (Gale Encyclopedia of Medicine, vol. 1, A–B) through a Streamlit chat, with agents that evaluate, summarise and escalate.



Planner

Haiku 5.5

Intent, query rewrite, accept / re-query / escalate



RAG

Opus 5.5

Hybrid retrieval and a cited, grounded answer



Evaluator

Opus 5.5

Fact-checks every claim, scores, revises



Summarizer

Haiku 5.5

Running memory every 3 turns, recaps

637

PDF pages ingested

282

encyclopedia entries (A–B)

3,328

structure-aware chunks

~$0.05

per medical question

"Tujuannya: menjawab pertanyaan medis dari Medical Book, yaitu Gale Encyclopedia of Medicine volume 1, topik A sampai B, lewat chat Streamlit.

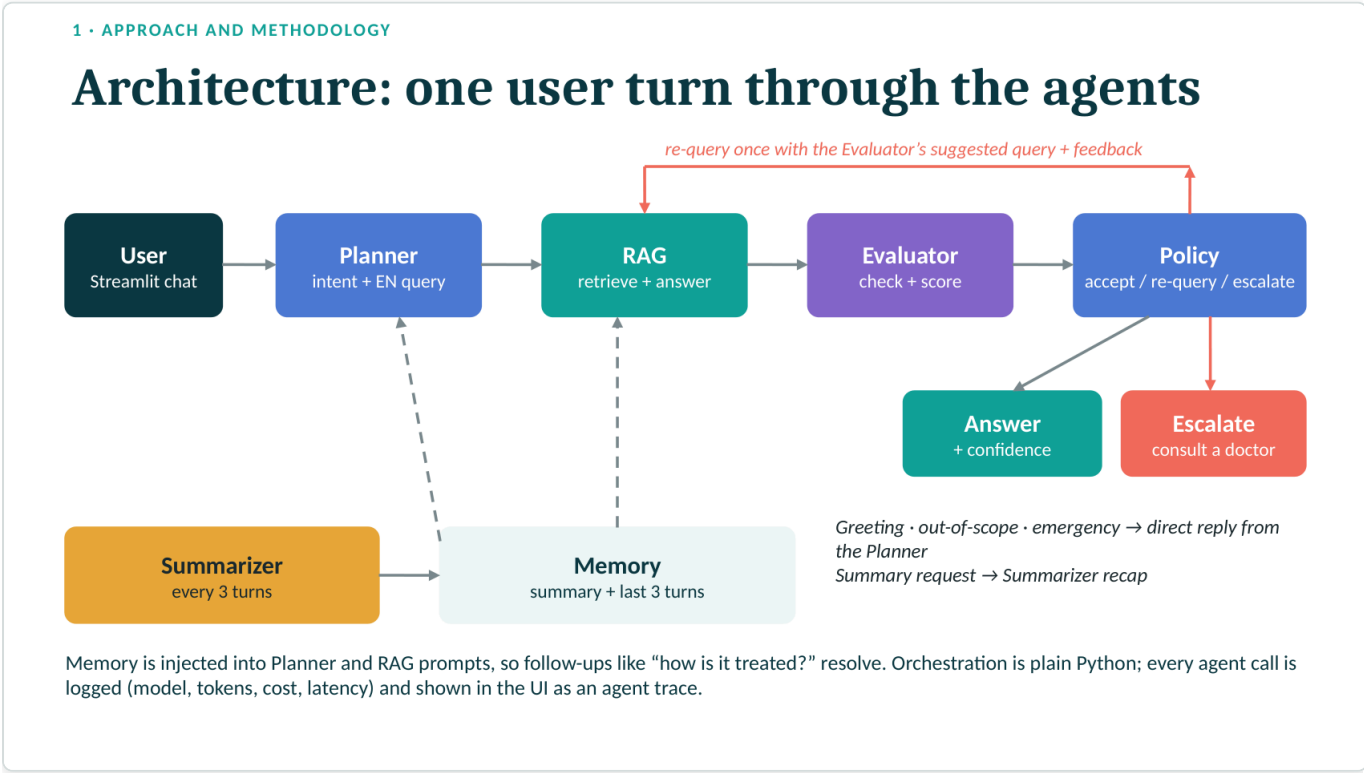
Saya membagi tugasnya ke **empat agent**, masing-masing dengan satu peran:

- **Planner** memahami maksud user, menulis ulang pertanyaan, dan memutuskan langkah berikutnya.
- **RAG** mencari potongan buku dan menulis jawaban bersitasi.
- **Evaluator** memeriksa setiap klaim di jawaban.
- **Summarizer** menjaga memori percakapan.

Planner dan Summarizer memakai **Haiku** yang murah dan cepat, karena tugasnya ringan. RAG dan Evaluator memakai **Opus**, karena di bidang medis ketelitian adalah yang terpenting.

Angka di bawah: **637 halaman** diolah menjadi **3.328 potongan** dari **282 topik**, dengan biaya sekitar **5 sen dolar per pertanyaan**."

Architecture: one user turn through the agents



"Ini alur satu pertanyaan. *[tunjuk dari kiri ke kanan]*

User bertanya, lalu **Planner** menentukan maksudnya dan menulis query bahasa Inggris. **RAG** mencari dan menjawab, **Evaluator** memeriksa dan memberi skor, lalu **Policy** memutuskan: **terima**, **cari ulang**, atau **eskalasi**.

Garis merah di atas adalah **loop cari ulang**: kalau jawabannya kurang meyakinkan, sistem mencari sekali lagi dengan query yang disarankan Evaluator.

Tidak semua pertanyaan melewati RAG. Sapaan, pertanyaan di luar topik, dan kondisi **darurat** langsung dijawab Planner. Untuk darurat, jawabannya langsung mengarahkan ke 119 atau IGD.

Di bawah ada **Summarizer**, yang setiap 3 giliran merangkum percakapan ke memori. Karena itu pertanyaan lanjutan seperti 'bagaimana cara mengobatinya?' tetap dipahami.

Orkestrasinya saya tulis dengan **Python biasa, tanpa framework**, supaya transparan dan mudah dites. Setiap panggilan agent tercatat dan bisa dilihat di UI."

Step 1 · Chunk along the book's own structure

1 · APPROACH AND METHODOLOGY

Step 1 · Chunk along the book’s own structure

ACNE
15 pt bold → entry

Causes and symptoms
11 pt bold → section

Acne vulgaris affects more than 85% of teenagers…
10 pt → body text

PyMuPDF reads font size and weight of every block; two-column pages are re-ordered; hyphenation is repaired; bibliographies are skipped.

Why not fixed-size chunks?

A chunk never mixes two diseases, and each one knows its entry, section and printed page, which become citations in the UI.

Parameters

- Max 1,200 characters per chunk (~300 tokens)
- 200-character overlap on sentence boundaries
- “Key terms” boxes kept as their own chunks
- 268 cross-references (“Abrasions see Wounds”) kept as query aliases

"Langkah pertama: memotong buku.

Cara umum adalah memotong setiap N karakter, tapi akibatnya satu potongan bisa berisi akhir artikel 'Acne' dan awal artikel penyakit lain. Saya memanfaatkan fakta bahwa **ensiklopedia punya pola font yang konsisten**: judul penyakit 15pt bold, sub-bagian 11pt bold, dan isi 10pt.

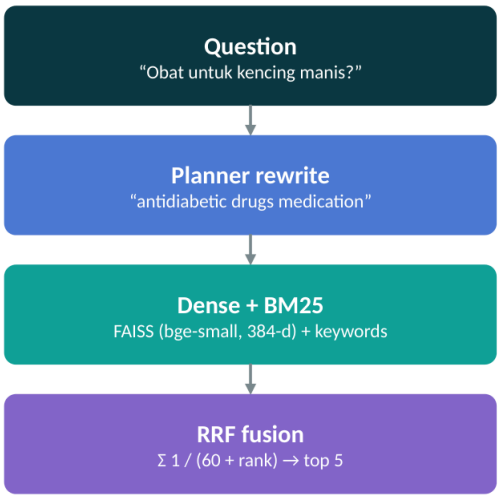
Dengan membaca **ukuran font** lewat PyMuPDF, buku dipotong **mengikuti strukturnya sendiri**. Hasilnya, satu potongan tidak pernah mencampur dua penyakit, dan setiap potongan tahu asalnya: penyakit apa, bagian apa, halaman berapa. Itulah yang nanti menjadi **sitasi** di jawaban.

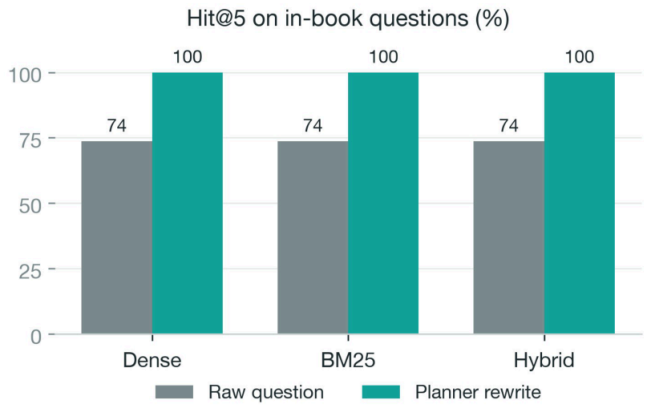
Parameternya: maksimal 1.200 karakter per potongan dengan overlap 200 karakter, dan 268 rujukan silang di buku disimpan sebagai alias."

Step 2 · Hybrid retrieval with query rewriting

1 · APPROACH AND METHODOLOGY

Step 2 · Hybrid retrieval with query rewriting





The book is English; users write Indonesian and lay terms. Rewriting the query is what makes retrieval work for them.

"Langkah kedua: mencari potongan yang relevan.

Saya memakai **hybrid search**: pencarian berdasarkan **makna** dengan FAISS dan embedding, digabung dengan pencarian **kata kunci** dengan BM25. Keduanya disatukan dengan **Reciprocal Rank Fusion**.

Tapi ada tantangan: **bukunya berbahasa Inggris**, sedangkan user menulis dalam **bahasa Indonesia** dan bahasa awam, misalnya 'obat kencing manis'. Karena itu Planner **menulis ulang pertanyaan** menjadi query medis bahasa Inggris, yaitu 'antidiabetic drugs'.

Dampaknya terlihat di chart: dengan pertanyaan mentah, Hit@5 hanya **74%**. Setelah ditulis ulang, **100%** di ketiga metode. Jadi **query rewriting adalah kunci** retrieval lintas bahasa."

Step 3 · Evaluate every answer, then decide

1 · APPROACH AND METHODOLOGY

Step 3 · Evaluate every answer, then decide

Evaluator (LLM-as-judge)

- Splits the answer into claims and checks each against the same excerpts the RAG agent saw
- Scores faithfulness, completeness, relevance, tone (0–1)
- Verdict: pass · revise (writes a corrected answer) · insufficient context (suggests a better query)
- JSON via structured outputs, validated with Pydantic

confidence = 0.5·F + 0.3·C + 0.2·R
computed in code; capped at 0.4 when the book lacks the answer

Planner policy (rules, not an LLM: free, predictable, unit-tested)

≥ 0.70	Accept and show the answer
< 0.70	Re-query once with feedback
0.45–0.70	After retry: accept with a warning
< 0.45	Escalate: “please consult a doctor”

“Insufficient context” twice → always escalate.

"Langkah ketiga: memeriksa dan memutuskan.

Evaluator bekerja sebagai *LLM-as-judge*. Ia memecah jawaban menjadi klaim-klaim dan mengecek setiap klaim terhadap potongan buku yang **sama** dengan yang dilihat RAG. Ia memberi skor **faithfulness, completeness, relevance, dan tone**, lalu verdict: *pass*, *revise* (yang berarti ia menulis jawaban yang diperbaiki), atau *insufficient context*.

Confidence dihitung di kode, bukan ditebak LLM: 0,5 faithfulness ditambah 0,3 completeness ditambah 0,2 relevance. Faithfulness bobotnya terbesar karena di bidang medis, jawaban **salah** lebih berbahaya daripada jawaban **kurang lengkap**.

Lalu Planner memutuskan dengan **aturan sederhana**, bukan LLM, supaya gratis, bisa diprediksi, dan bisa dites: di atas 0,70 diterima; di bawahnya dicari ulang sekali; kalau masih di bawah 0,45 atau buku memang tidak membahas, maka **eskalasi ke dokter**."

Model architecture and parameters

2 · MODEL ARCHITECTURE AND PARAMETERS

Model architecture and parameters

Component	Choice	Parameters
Ingestion	PyMuPDF, font-aware parsing	1,200 chars, 200 overlap, 3,328 chunks
Embeddings	BAAI/bge-small-en-v1.5 (fastembed, ONNX, CPU)	384-d, normalised, local and free
Vector index	FAISS IndexFlatIP	exact cosine search, top-30 candidates
Keyword search	BM25 (rank-bm25)	top-30 candidates, alias expansion
Fusion	Reciprocal Rank Fusion	k = 60, entry-title boost, top-5 to the LLM
Planner	claude-haiku-5-5	effort low, structured output (intent, query)
RAG	claude-opus-5-5	effort low, max 4k tokens, refusal fallback
Evaluator	claude-opus-5-5	effort medium, structured output (scores, verdict)
Summarizer	claude-haiku-5-5	effort low, every 3 turns, ≤120 words
Policy	rules in Planner	accept ≥ 0.70 · escalate < 0.45 · max 2 attempts

"Ini ringkasan semua komponen dan parameternya. *[jangan dibaca satu per satu, sorot yang penting]*

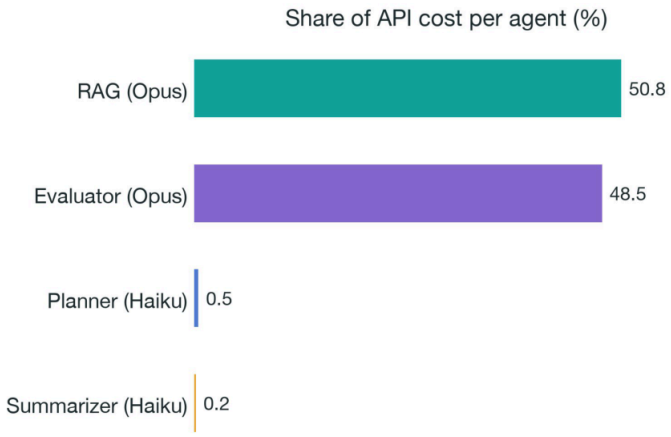
Beberapa yang ingin saya sorot:

- **Embedding bge-small berjalan lokal di CPU**, jadi gratis dan tanpa GPU.
- **FAISS IndexFlatIP**, yaitu pencarian exact. Untuk 3.300 vektor ini sudah lebih dari cukup; index aproksimasi baru perlu untuk jutaan dokumen.
- **Top-5** potongan dikirim ke LLM.
- Semua parameter ini ada di **satu file config**, jadi mudah di-tuning."

A cost-aware model mix

2 · MODEL ARCHITECTURE AND PARAMETERS

A cost-aware model mix



<1%

of the cost goes to routing and memory (Haiku 5.5)

$0.050

average per medical question (eval run)

Accuracy-critical steps (answering, fact-checking) use the strongest model; light classification and summarising use the cheapest. Measured on a 4-question smoke test; every model is one line in config.py.

"Kenapa kombinasi Haiku dan Opus? Chart ini menunjukkan **sekitar 99% biaya ada di RAG dan Evaluator**, tahap yang paling butuh ketelitian. Planner dan Summarizer dengan Haiku hanya memakan **kurang dari 1%**.

Rata-rata biayanya **5 sen dolar per pertanyaan medis**. Prinsipnya: **model terkuat untuk tahap yang menentukan akurasi, model termurah untuk tugas ringan**, dan setiap model bisa diganti di satu baris config."

Evaluation set: 27 questions, every behaviour

3 · EVALUATION METRICS AND RESULTS

Evaluation set: 27 questions, every behaviour

- 8 In-book, English
- 7 In-book, Indonesian
- 4 Lay terms (EN/ID)
- 4 Out-of-book medical
- 2 Non-medical
- 1 Emergency
- 1 Greeting

Out-of-book topics (dengue, COVID-19, Zika, semaglutide) were verified to appear nowhere in the book, so the right behaviour is to escalate.

Routing
Planner intent accuracy

Retrieval
Hit@5 and MRR of the expected entry

Answer quality
Faithfulness, completeness, relevance, tone, valid citations

Safety
In-book answered vs. out-of-book escalated

Efficiency
Latency and USD per question

"Untuk membuktikan sistemnya bekerja, saya membuat **27 pertanyaan uji** yang mencakup **semua perilaku**: pertanyaan yang ada di buku dalam bahasa Inggris dan Indonesia, bahasa awam, pertanyaan **di luar buku**, non-medis, darurat, dan sapaan.

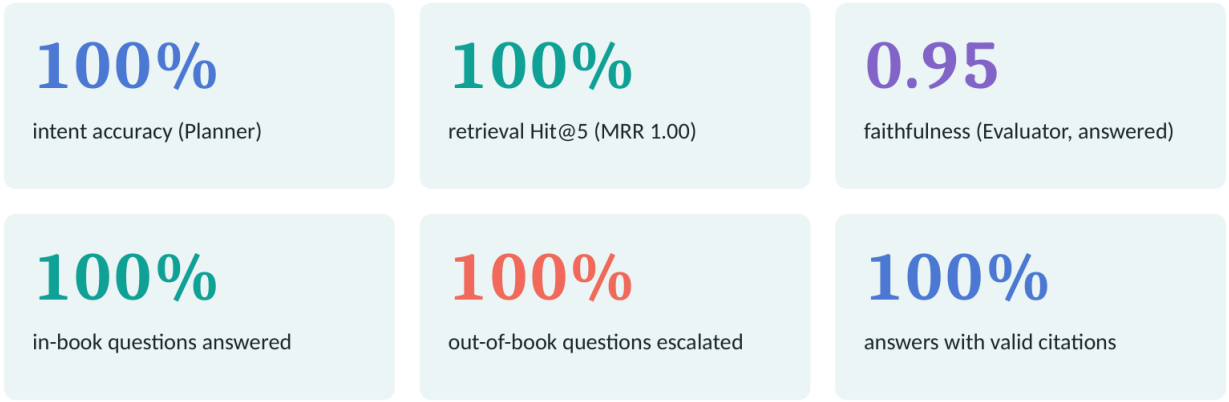
Untuk pertanyaan di luar buku, saya memilih topik seperti dengue, COVID-19, Zika, dan semaglutide, lalu **memastikan topik itu benar-benar tidak ada di buku**. Jadi jawaban yang benar untuk keempatnya adalah **eskalasi, bukan menjawab**.

Metriknya mencakup lima aspek: **routing, retrieval, kualitas jawaban, keamanan, dan efisiensi**."

Results: grounded, accurate, safe on unknowns

3 · EVALUATION METRICS AND RESULTS

Results: grounded, accurate, safe on unknowns



Completeness 0.89 · relevance 0.95 · tone 0.94 · avg confidence 0.93 in-book vs 0.40 out-of-book · latency 13.5 s avg (p90 20.9 s) · whole run $1.16

"Hasilnya:

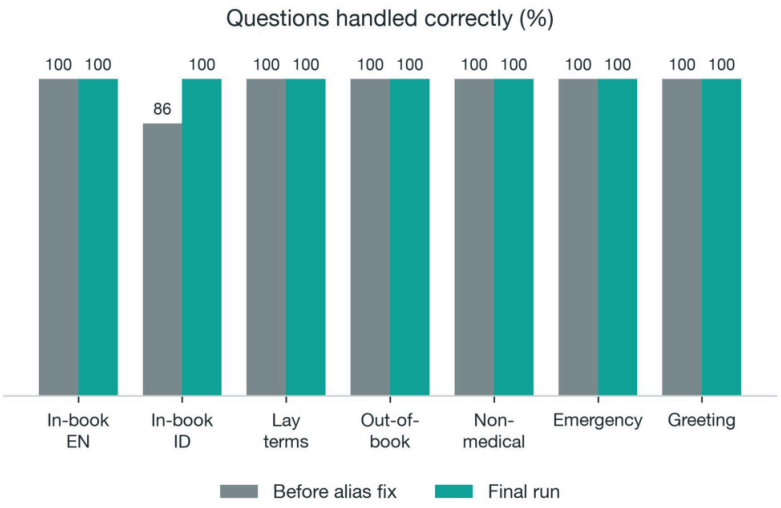
- **100%** intent accuracy: Planner selalu benar membedakan jenis pertanyaan.
- **100%** retrieval Hit@5: entri buku yang benar selalu ditemukan.
- **Faithfulness 0,95**: hampir semua klaim didukung sumber.
- **100%** pertanyaan di buku dijawab, dan yang terpenting, **100% pertanyaan di luar buku dieskalasi**. Sistem **tidak mengarang**.
- **100%** sitasi valid.

Confidence rata-rata **0,93** untuk pertanyaan yang ada di buku, dibanding **0,40** untuk yang di luar buku. Artinya skor confidence benar-benar membedakan keduanya. Seluruh evaluasi memakan biaya **$1,16**."

Correct behaviour by question category

3 · EVALUATION METRICS AND RESULTS

Correct behaviour by question category



What "correct" means

- In-book: answered, not escalated
- Out-of-book: escalated, nothing invented
- Others: routed to the right intent

100%

of all 27 questions handled correctly

In-book ID: 86% → 100%

The first run exposed a retrieval bug (alias "ALS" matched inside "palsy"); fixing it made every category correct.

"Per kategori, di run final **semua 27 pertanyaan ditangani dengan benar**.

Perhatikan batang abu-abu di 'In-book ID': **86%**. Itu hasil **run pertama**, dan saya akan menceritakannya di slide berikutnya, karena justru evaluasi itulah yang menemukan bug di sistem saya."

Observations and challenges

4 · OBSERVATIONS AND CHALLENGES

Observations and challenges

The evaluation caught a real bug

Alias “ALS” matched inside “p-als-y”, so Bell’s palsy retrieved ALS and was wrongly escalated. Fixed with whole-word matching plus a regression test.

Cross-lingual retrieval

English embeddings miss Indonesian and lay terms (“kencing manis”). The Planner’s English query rewrite fixes this (see ablation).

Judging with an LLM

Evaluator scores are model judgements and can be lenient. Mitigated with a strict rubric, claim-level checks and code-computed confidence; human labels are next.

Latency vs. safety

~14 s per answer: two sequential Opus calls (answer, then check). A re-query doubles it; streaming would hide most of the wait.

"Ada empat temuan:

Pertama, evaluasi menemukan bug nyata. Di run pertama, pertanyaan tentang 'Bell's palsy' malah dieskalasi. Ternyata ekspansi alias memakai pencocokan *substring*, sehingga singkatan **'ALS' cocok di dalam kata 'p-ALS-y'**, dan hasil pencarian tercemar artikel ALS. Saya memperbaikinya dengan pencocokan **kata utuh** dan menambahkan **regression test**. Pelajarannya: evaluasi bukan formalitas, karena ia menemukan bug yang tidak terlihat saat demo.

Kedua, retrieval lintas bahasa. Embedding bahasa Inggris kesulitan dengan bahasa Indonesia dan bahasa awam, dan solusinya adalah query rewriting oleh Planner.

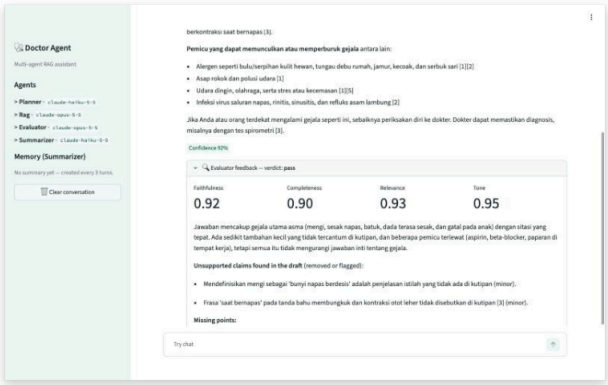
Ketiga, keterbatasan LLM-as-judge. Skor Evaluator tetap penilaian model dan bisa terlalu lunak. Saya memitigasinya dengan rubrik ketat, pengecekan per klaim, dan confidence yang dihitung di kode. Langkah berikutnya adalah penilaian manual oleh tenaga medis.

Keempat, latensi versus keamanan. Satu jawaban butuh sekitar **14 detik**, karena ada dua panggilan Opus berurutan: menjawab, lalu memeriksa. Ini konsekuensi yang saya pilih demi keamanan. Untuk mengurangi rasa menunggu, UI menampilkan progres agent secara langsung."

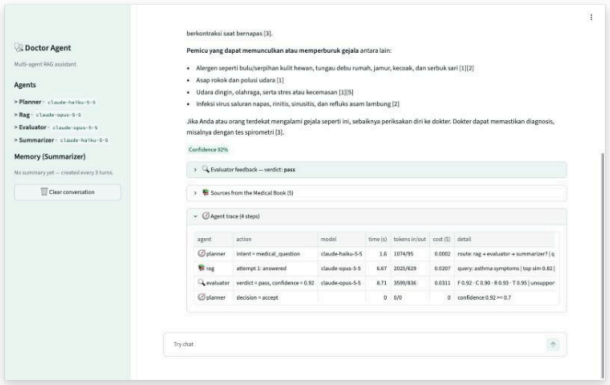
Streamlit UI: a checked, cited answer

5 · SCREENSHOTS OF THE STREAMLIT INTERFACE

Streamlit UI: a checked, cited answer



Indonesian question, English book: 92% confidence, Evaluator scores and checked claims



Agent trace: model, tokens, cost and latency per step

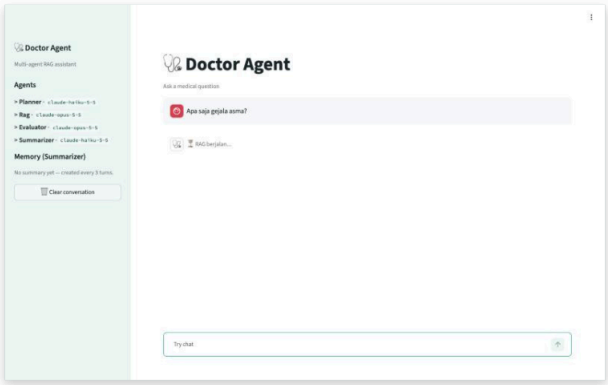
"Ini tampilan aplikasinya. *[tunjuk kiri]* Pertanyaan bahasa Indonesia 'apa saja gejala asma?' dijawab dalam bahasa Indonesia dari buku berbahasa Inggris, lengkap dengan **sitasi**, badge **Confidence 92%**, dan panel **Evaluator feedback** berisi empat skor serta klaim yang dicek.

[tunjuk kanan] Panel **Agent trace** menunjukkan kerja setiap agent: Planner, RAG, Evaluator, lalu keputusan Planner, lengkap dengan model, token, biaya, dan waktunya. Di sinilah kolaborasi multi-agent-nya **terlihat langsung**."

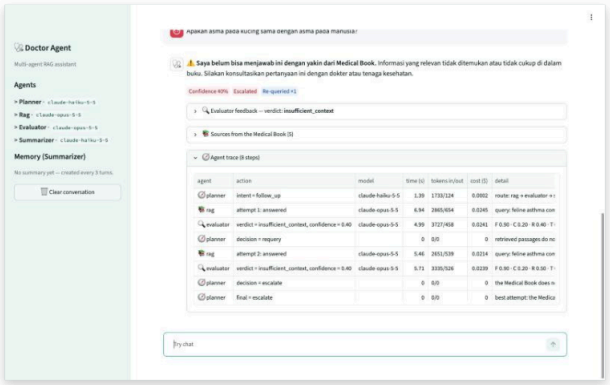
Streamlit UI: live progress and an escalation

5 · SCREENSHOTS OF THE STREAMLIT INTERFACE

Streamlit UI: live progress and an escalation



One live progress line while the agents work ("RAG berjalan...")



Asthma in cats is not in the book → re-query → escalate

"*[tunjuk kiri]* Selama proses, user melihat **progres langsung**: 'Planner berjalan', lalu 'RAG berjalan', lalu 'Evaluator berjalan'. Begitu jawaban siap, tulisan itu hilang.

[tunjuk kanan] Ini contoh **eskalasi**. Pertanyaan 'apakah asma pada kucing sama dengan asma pada manusia?' tidak ada di buku, karena buku ini ensiklopedia medis manusia. Sistem **mencari ulang sekali**, tetap tidak menemukan, lalu **jujur mengatakan tidak bisa menjawab** dan menyarankan konsultasi, alih-alih mengarang jawaban.

Sebagai penutup: Doctor Agent **menjawab dari sumber, memeriksa dirinya sendiri, dan jujur saat tidak tahu**. Ketiga sifat ini menurut saya yang paling penting untuk AI di bidang medis. Terima kasih, saya siap menerima pertanyaan."

Tips Saat Presentasi

- **Jangan membaca slide.** Pakai naskah ini sebagai panduan, lalu ucapkan dengan kata-katamu sendiri.
- **Siapkan demo live** setelah slide 15 dengan tiga pertanyaan berurutan:
  1. "Apa itu asma?" → alur terima, confidence tinggi
  2. "Bagaimana cara mengobatinya?" → follow-up dipahami, Summarizer mulai bekerja
  3. "Apakah asma pada kucing sama dengan asma pada manusia?" → cari ulang lalu eskalasi
- **Kalau waktunya terbatas** (misalnya 10 menit), persingkat slide 8 dan 10, lalu fokus di slide 4, 6, 7, 11, dan 13.
- **Angka yang perlu dihafal:** 4 agent · 637 halaman → 3.328 potongan · Hit@5 74% → 100% · faithfulness 0,95 · 100% di luar buku dieskalasi · ±$0,05 dan ±14 detik per pertanyaan.